

Project — Learning Worlds



1 Objective

Working **individually or in pairs**, design a small interactive world in which one or more autonomous agents learn a behavior using deep learning. You are free to choose the environment, rules, observations, actions, objective, model, and training method.

The goal is not necessarily to produce a perfect agent, but to demonstrate a coherent process: environment design, learning, experimentation, evaluation, and critical analysis. Possible topics include cellular automata, survival or exploration games, Snake, a simplified Pac-Man or Tower Defense, artificial ecosystems, and multi-agent simulations. Other original ideas are welcome, subject to approval.

2 Minimum Requirements

The project must include:

- a dynamic environment with a visual representation;
- at least one autonomous agent;
- clearly defined observations, actions, and objectives;
- a trained deep learning model;
- a quantitative evaluation protocol;
- a comparison with at least one non-deep-learning baseline agent.

The baseline may be random, rule-based, heuristic, manually controlled, or based on a classical algorithm. The learned agent may use supervised learning, imitation learning, reinforcement learning, self-play, evolutionary learning, or a justified combination of methods.

You may develop the environment from scratch or build upon an existing engine, library, or open-source project. In all cases, clearly identify your own contributions and any external resources used.

If you need inspiration or a starting point, you may consult the *TinyWorld* repository used in the course demonstrations ([link](#)). The repository is provided for inspiration only and is not the expected solution.

3 Expected Work

Your work should cover the following stages:

1. define the environment, observations, actions, and objective;
2. implement a functional environment and a baseline agent;
3. design and train a deep learning model;
4. define evaluation metrics and conduct several experiments;
5. compare the learned agent with the baseline;
6. analyze the observed behaviors, limitations, and possible improvements.

Formulate at least one hypothesis and evaluate it experimentally by modifying a single aspect of your approach while keeping the others unchanged. It can be the influence of the DL architecture, observation representation, reward function, training duration, memory, environment diversity, or model depth. You may also study robustness and generalization to unseen situations.

Present the results with appropriate evidence: plots, tables, statistics, screenshots, or videos.

4 Deliverables

Submit a link to a Git repository containing:

- the complete source code;
- a **README** with installation, execution, and usage instructions;
- the commands required to run a demonstration and reproduce the evaluation;
- the trained model weights, when their size permits, or instructions to download them;
- a concise report;
- a short video or replay demonstrating the learned behavior.

The repository should be self-contained and executable on another machine without significant modifications.

5 Report and Presentation

The report (maximum 2 pages) should briefly describe:

- the problem and environment;
- observations, actions, and baseline agent;
- the model and learning method;
- the experimental protocol and results;
- the interpretation of behaviors, limitations, and future improvements.

Do not merely describe the code: justify your design choices and interpret the results.

The presentation must include the project concept, a demonstration, the model and training method, a comparison with the baseline, and a discussion of the results and limitations. The demonstration may be live or prerecorded.

6 Assessment

Criterion	Weight
Environment design, originality, and integration	15 %
Relevance of the model and learning method	25 %
Quality of the experimental protocol	20 %
Analysis and interpretation of the results	20 %
Technical quality and reproducibility	10 %
Presentation and demonstration	10 %

Raw performance alone is not sufficient. An imperfect agent can receive an excellent grade if the methodology is rigorous, the experiments are relevant, and the limitations are analyzed honestly. Creativity, scientific rigor, and the quality of the analysis are considered more important than achieving the highest score.

Happy coding!